Post-Quantum Cryptography Migration

Part 9 - Building & Shipping Software: Software Supply Chain

Compiled by the Spaced Research Ledger

Last updated: 2026-08-29

Every piece of software you install was vouched for by a signature: the package manager checked it, the app store checked it, the container runtime checked it. Those signatures are almost all classical, and unlike a TLS session they must stay valid for years. Forging one retroactively would poison the supply chain at its source.

This report covers the signing infrastructure in four sections. Section 1 covers the artifact-signing frameworks, Section 2 the package ecosystems, Section 3 the platform code-signing regimes, and Section 4 container signing and the CI/CD pipelines.

Two real firsts landed this year. Red Hat began signing production RPM packages with a hybrid post-quantum key, calling itself the first major Linux distribution to do so. And Android 17 ships a hybrid APK signature scheme pairing a classical key with ML-DSA, enforced on new devices while old ones ignore it.

The center of the ecosystem still waits on one dependency. Sigstore, which signs npm packages, Python attestations, and GitHub's build provenance, has tied its public instance's migration to ML-DSA landing in Go's standard library. That package is now in a release candidate, which is why the second half of 2026 keeps appearing in every plan.

1. Artifact Signing Frameworks

A few frameworks define how modern software gets signed. Sigstore issues short-lived certificates tied to identities and logs every signature publicly. TUF secures update systems with layered signing roles. Notary signs container artifacts. What these projects decide about post-quantum algorithms flows downstream into every ecosystem built on them.

Recent Developments

The frameworks' specifications remain classical, deliberately. The current TUF specification defines only RSA, Ed25519, and ECDSA as recommended, while stating the format is architecturally open to any scheme [1]. The action is in Sigstore's engineering, described below, where the crypto-agility groundwork is complete and the algorithm decisions are being contested in public.

Current State

Sigstore has done the plumbing ahead of the policy. Its leadership's roadmap commits to post-quantum signing as soon as possible, while gating the shared public services on vetted implementations in Go's standard library [2]. It chose ML-DSA over SLH-DSA for its smaller signatures [2], and its project policy selects both for experiments while excluding stateful schemes from ephemeral use [3]. Trail of Bits spent roughly two years building the agility: a centralized algorithm registry, server flags restricting accepted algorithms,

and a client signing-algorithm flag [4]. The work was demonstrated end to end with working Go implementations of LMS and ML-DSA [4].

Conference talks show two prototype paths: a hybrid transparency log roughly doubling bundle size, and hybrid certificate extensions at the CA layer [5]. They name the codebase's hardcoded algorithm assumptions as the structural obstacle [5]. Cloud key services are ready when Sigstore is, with AWS signing ML-DSA since mid-2025 [6]. A community proof of concept already signs and verifies with ML-DSA-65 on a patched Go build [7]. The registry lists the ML-DSA variants as experimental and not yet functional [8].

TUF and Notary are the quiet dependencies. Researchers modeled TUF's four-role signature chain under post-quantum sizes [9], recommending stateful hash-based schemes for update security [9], and peer-reviewed work maps phased migrations for TUF and Notary hierarchies [10][11]. Sigstore itself flags an unresolved question: its own trusted-root distribution runs on TUF, and nobody has settled how that material accommodates hybrid keys [2]. Notary's specification still defines only RSA and ECDSA, with no post-quantum entry [12]. In the background sits the standards history: the 2024 finalization gave these ecosystems their first stateless targets after years when hash-based schemes were the only vetted option [2].

Unresolved Conflicts

Sigstore's timeline is told two ways. One source asserts its Technical Advisory Committee formally approved ML-DSA with certificates flowing in 2025 to 2026 and full support in the second half of 2026 [13]. The project's own roadmap describes public-instance adoption as gated on the Go standard library, with open design questions [2].

A dated statement from the project would settle it. Two related disputes ride along. One is whether the public instance moves before or after that library milestone [2]. The other is whether lattice or stateful hash-based signatures are the right long-term supply-chain choice at all [14].

2. Package Ecosystem Signing

Package managers are how code actually reaches machines: apt and rpm on servers, PyPI and npm for developers, Maven for the Java world. Each has its own signing lineage, from decades-old PGP to Sigstore-born attestations. This section covers the first production post-quantum packages, and the OpenPGP standard that finally makes them official.

Recent Developments

OpenPGP got its post-quantum RFC. RFC 9980, published June 2026, assigns signature algorithm identifiers for composite ML-DSA with Edwards curves and for the SLH-DSA variants [15]. It confines all post-quantum signatures to the new version-6 keys, with only the encryption subkey allowed on legacy keys [15]. The gap is tooling: GnuPG ships post-quantum encryption but reportedly no post-quantum signatures, exactly the piece package verification needs [16].

Python's packaging world gained the primitives, with ML-KEM and ML-DSA shipped into its main cryptography library [17], though the current attestation signature remains classical ECDSA [18]. Sigstore is

deliberately holding its public log on the stable version to minimize disruption ahead of post-quantum breaking changes [19].

Current State

Red Hat crossed the line first. It modernized its signing infrastructure, created a hybrid ML-DSA-87 plus Ed448 key, and began dual-signing RHEL packages, starting with a named package that also keeps its RSA signature [20]. It funded the post-quantum work in Sequoia-PGP and shipped hardware-backed signing support [20]. Verification shows both signature generations side by side, with guidance to trust both certificates so either algorithm's compromise survives [21].

Downstream rebuilds inherit the capability, with AlmaLinux and Oracle Linux documenting the new signature format and generation commands [22][23]. Debian and Ubuntu, by contrast, show no post-quantum archive-signing plan, only routine classical key rotation [24].

The developer ecosystems ride Sigstore's schedule. PyPI's attestations are identity-bound statements signed through Sigstore, not maintainer keys [25]. PGP uploads have been disabled there since 2023, so the OpenPGP RFC has nothing to inherit into PyPI [26]. npm's provenance flows through the public Sigstore servers [27], which historically hardcoded one classical algorithm until the agility work landed [4].

Those servers added ML-DSA to the specification [2] and wait on the Go library for the services themselves [2]. Maven Central still requires detached PGP signatures [28] while accepting Sigstore bundles alongside [29]. The Java platform is ready above it, with JDK 26 signing and verifying ML-DSA JARs [30] and a community plugin doing hybrid Maven signing today [31].

The Go dependency is nearly resolved, with caveats. Go 1.26 carries an internal ML-DSA implementation, with the public package proposed for 1.27 [32]. The package is excluded from the certified FIPS module until a later version, constraining FIPS-mode deployments [33]. And the GnuPG signature gap rests on secondary sources, with a comparison page corroborating that the algorithms are reachable only through an experimental engine [34].

Unresolved Conflicts

Red Hat's own status labels disagree. One page classifies post-quantum RPM signing as a technology preview not for production [35], while another describes it as remaining in production with dual signatures [36]. The likeliest reading is a preview feature running in Red Hat's own production signing, but the sources do not say so. A Red Hat support statement would settle it.

3. Platform Code Signing

Operating systems enforce their own signing regimes: Authenticode on Windows, notarization on macOS, APK signatures on Android. These are the gates malware must pass, and their algorithms are baked into billions of devices. This section covers the first hybrid platform scheme shipping on Android, Windows moving in stages, and Apple saying nothing.

Recent Developments

Android shipped the first hybrid platform signature. Android 17's new APK scheme pairs a classical key with ML-DSA, backward-compatible because older versions verify only the classical half, and requiring a fresh classical key rather than reuse [37]. The published specification pins the details: exactly two signers, the larger ML-DSA parameter sets, stripping protection, and version-gated verification rules [38].

Windows moved the certificate layer and set a date. ML-DSA certificate issuance is generally available, in all parameter sets, with size tradeoffs left to the operator [39]. Composite algorithm support reached current Windows through July updates [39]. Consulting analysis calls the release end-to-end for the signing plane, including Authenticode with ML-DSA, with no in-place migration and no backports [40].

Microsoft's servicing documentation says Windows production signing transitions to post-quantum in 2027, possibly with hybrids [41]. Apple, meanwhile, is described by third parties as a classical Developer ID plus notarization pipeline, with post-quantum signing an industry gap rather than an Apple option [42].

Current State

Android's rollout is deliberately staged. Google announced the platform will verify post-quantum APK signatures so installs resist quantum-era forgery [43], with beta testing ahead of the production release [43]. Play documentation names the hybrid block and its opt-in at key upgrade [44]. Enforcement tiers by version, strict on Android 17, classical-only in the middle, and unenforced on the oldest [44], with one compatibility exclusion around the newest distribution format [44].

Google Play generates the ML-DSA keys through its cloud key service during the rollout, with developer-owned keys later and rotation prompts every two years [43]. The surrounding platform moved in the same release, with verified boot, attestation, and the key store all gaining ML-DSA [43].

Windows has working signatures and an unfinished public-trust story. The certificate service supports ML-DSA across every CA role and OCSP signing [45], templates can issue ML-DSA code-signing certificates with signing-only usage [46], and enrollment requires current clients [46]. Hands-on testing signed a PowerShell script with an ML-DSA certificate successfully while a TLS binding failed in the same session [47]. Publicly trusted ML-DSA remains in drafting at the CA/Browser Forum, where Microsoft backs the ballot and is piloting certificates while suggesting they stay out of transparency logs [48].

Separately, code-signing certificate lifetimes were capped at 460 days for unrelated reasons [49]. All of it builds on the primitive layer shipped in November 2025 [50].

Apple's silence is nearly total. Its concrete movement is at the bottom of the stack, publishing its verified ML-KEM and ML-DSA implementations [51]. Its own description of the post-quantum program lists messaging, VPN, TLS, and developer APIs, never code signing or notarization [52], continuing the emphasis it set in 2024 [53].

Unresolved Conflicts

Microsoft's code-signing timeline exists in three versions. An aggregator describes kernel-driver dual-signing acceptance starting 2025 to 2026 with named system binaries and a 2029 readiness target, unconfirmed by

any Microsoft primary source [54]. Microsoft's own servicing document says production signing transitions in 2027 [41]. And a vendor blog attributes ML-DSA Authenticode to a specific tool release and a Forum ballot number that does not match the working group's naming convention, both uncorroborated [55]. Only Microsoft's published plan carries primary authority, and a consolidated Microsoft timeline would settle the rest.

4. Container Signing & CI/CD

Modern build pipelines sign what they produce: container images through cosign, build provenance through GitHub's attestations, artifacts through cloud key services. The keys often live for minutes, but the verification infrastructure must last. This section covers the transparency logs already accepting post-quantum signatures, and the cloud services that got there first.

Recent Developments

The transparency-log layer moved before the signing layer. The checkpoint specification now supports ML-DSA, with logs recommended to use its smallest variant for cosignatures [56]. The Tessera log implementation accepts ML-DSA keys with classical deprecation planned, and ecosystem witnesses already return ML-DSA cosignatures [56][56]. Meanwhile the Go dependency arrived in release-candidate form: the standard-library ML-DSA package exists with all parameter sets and pre-hash signing [33].

Its published sizes confirm the bundle-bloat arithmetic, with signatures around 2.4 to 4.6 kilobytes [33]. cosign's native ML-DSA proposal is accepted and planned for late summer 2026 [57]. The public Sigstore instance stays on its stable log version to spare clients churn before the transition [58].

Current State

The CI platforms already run the target architecture. GitHub's artifact attestations use ephemeral in-memory keys, an identity-bound short-lived certificate, a timestamped envelope, and immediate key disposal [59], with private repositories served by an internal CA [60]. That design migrates by swapping algorithms, not architecture. Practitioners are not waiting.

A documented GitLab pipeline dual-signs artifacts with a classical Vault signature plus a lattice signature today [61], and commercial signing services advertise ML-DSA and LMS with pipeline integrations [62]. One gap is infrastructural rather than cryptographic: Azure DevOps's hosted Git SSH endpoint negotiates no post-quantum key exchange, and new OpenSSH clients warn about it [63].

The cloud key services are the mature layer. AWS KMS signs with all three ML-DSA parameter sets inside validated HSMs, with a size threshold above which callers pre-hash externally [6][64]. Its private CA issues ML-DSA certificates with code signing as a named use, across every region [65], and the pair is positioned as a quantum-resistant signing chain [66]. Google Cloud KMS reached general availability for all three NIST algorithms [67], offering ML-DSA and a compact SLH-DSA variant across the security categories [68].

It deliberately omits hybrid signatures pending community consensus [69]. Vault documents ML-DSA, SLH-DSA, and a hybrid key type pairing a lattice key with a classical curve [70], grown from an explicitly experimental start [71].

Sigstore's remaining work is choices, not code. The public instance waits on the vetted library [2], the specification carries ML-DSA for client-provided keys [2]. The agility layer is shipped, with out-of-band algorithm suites chosen specifically to avoid the confusion attacks of in-band signaling [4][4]. No post-quantum suite is in the shipped registry yet, and the demonstrations remain on the blob path rather than the container path [4][4]. Red Hat's prototypes quantify the hybrid cost at roughly double the bundle size, with legacy clients unharmed but unprotected [5].

The project's published open questions are the honest list [2]. They cover how the trusted root carries mixed material, whether services dual-return signatures without enabling stripping, and how users express policy over the mix [2]. The newer tile-backed log design trims entry types with client support in current tools [72]. Notary cannot participate at all yet, with both its signature and envelope specifications enumerating only classical algorithms [12][73]. Full first-class cosign support is expected in the second half of 2026 [74].

Unresolved Conflicts

Azure's key-service status is contested. One analysis says its vaults and managed HSM support only classical key types, with post-quantum requiring an external hardware path [75]. Another describes ML-KEM and ML-DSA arriving through the platform's cryptography library with tiered availability into 2027 [76]. Microsoft's own key-service documentation, checked directly, lists no ML-DSA key type and discusses quantum resistance only for symmetric keys [77]. The documentation is the closest thing to ground truth, and a Microsoft announcement would settle the rest.

About This Report

The Spaced Research Ledger is an automated system that gathers claims from live public sources, checks each one against its origin, and records every disagreement instead of merging it. Every stage runs on a different AI model, drawn from four to seven systems across competing labs, so no single model both finds a claim and judges it. The Spaced Research Ledger is designed and maintained by Ridley DiSiena.

Sources

1. github.com — tuf spec - https://github.com/theupdateframework/specification/blob/master/tuf-spec.md - as of 2026-07-15
2. blog.sigstore.dev — post quantum 2025 - https://blog.sigstore.dev/post-quantum-2025/ - as of 2025-06-06
3. github.com — POLICY - https://github.com/sigstore/sigstore/blob/main/POLICY.md
4. blog.trailofbits.com — building cryptographic agility into sigstore - https://blog.trailofbits.com/2026/01/29/building-cryptographic-agility-into-sigstore/ - as of 2026-01-29

5. tldrecap.tech — quantum proofing sigstore post quantum cryptography - https://tldrecap.tech/posts/2026/opensource-securitycon/quantum-proofing-sigstore-post-quantum-cryptography/ - as of 2026-03-24
6. aws.amazon.com — aws kms post quantum ml dsa digital signatures - https://aws.amazon.com/about-aws/whats-new/2025/06/aws-kms-post-quantum-ml-dsa-digital-signatures - as of 2025-06-13
7. cosign with post-quantum MLDSA sample · sigstore/cosign · Discussion #4771 - https://github.com/sigstore/cosign/discussions/4771 - as of 2026-03-16
8. architecture-docs/algorithm-registry.md at main · sigstore/architecture-docs - https://github.com/sigstore/architecture-docs/blob/main/algorithm-registry.md
9. arxiv.org — 2502.18092 - https://arxiv.org/abs/2502.18092 - as of 2025-02-25
10. Quantum-Resilient PKI for Container Supply Chains: Post-Quantum Cryptography in TUF and Notary Frameworks - https://www.ijirset.com/upload/2025/november/187_Quantum-Resilient%20PKI%20for%20Container%20Supply%20Chains%20Post-Quantum%20Cryptography%20in%20TUF%20and%20Notary%20Frameworks.pdf - as of 2025-11-01
11. Post-Quantum Software Updates with Stateful Hash-Based Signatures and The Update Framework - https://www.computer.org/csdl/magazine/sp/2026/02/11111737/28UtqfViOWc - as of 2026-03-01
12. github.com — signature specification - https://github.com/notaryproject/specifications/blob/main/specs/signature-specification.md
13. Post-Quantum Artifact Signing in CI/CD: Migrating cosign and Sigstore to ML-DSA - https://www.systemshardening.com/articles/cicd/post-quantum-artifact-signing/ - as of 2026-05-08
14. postquantum.com — signature supply chain - https://postquantum.com/post-quantum/signature-supply-chain/ - as of 2026-05-03
15. datatracker.ietf.org — rfc9980 - https://datatracker.ietf.org/doc/rfc9980/ - as of 2026-06-30
16. postquantum.com — post quantum openpgp rfc 9980 - https://postquantum.com/security-pqc/post-quantum-openpgp-rfc-9980/ - as of 2026-07-11
17. blog.trailofbits.com — shipping post quantum cryptography to python - https://blog.trailofbits.com/2026/06/30/shipping-post-quantum-cryptography-to-python/ - as of 2026-06-30
18. pydevtools.com — what is pep 740 - https://pydevtools.com/handbook/explanation/what-is-pep-740/ - as of 2026-06-24
19. blog.sigstore.dev — rekor evolution - https://blog.sigstore.dev/rekor-evolution/ - as of 2026-06-28
20. redhat.com — whats new post quantum cryptography rhel 101 - https://www.redhat.com/en/blog/whats-new-post-quantum-cryptography-rhel-101 - as of 2026-02-17
21. docs.redhat.com — verifying rpm packages with post quantum signatures security hardening - https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/security_hardening/verifying-rpm-packages-with-post-quantum-signatures_security-hardening
22. wiki.almalinux.org — 10.1 - https://wiki.almalinux.org/release-notes/10.1.html - as of 2025-11-24
23. docs.oracle.com — ol10 features Security - https://docs.oracle.com/en/operating-systems/oracle-linux/10/relnotes10.1/ol10-features-Security.html - as of 2026-01-26

24. ftp-master.debian.org — keys - https://ftp-master.debian.org/keys.html
25. docs.pypi.org — attestations - https://docs.pypi.org/attestations/
26. peps.python.org — pep 0740 - https://peps.python.org/pep-0740/
27. docs.npmjs.com — generating provenance statements - https://docs.npmjs.com/generating-provenance-statements/
28. central.sonatype.org — 20220310 sigstore - https://central.sonatype.org/news/20220310_sigstore/
29. central.sonatype.org — 20250128 sigstore signature validation via portal - https://central.sonatype.org/news/20250128_sigstore_signature_validation_via_portal/ - as of 2025-01-28
30. Release Note: Signed JAR Support for ML-DSA - https://bugs.openjdk.org/browse/JDK-8371578 - as of 2026-01-27
31. io.github.aloubyansky.pqc.maven:pqc-sign-maven-plugin 0.0.2 on Maven - https://libraries.io/maven/io.github.aloubyansky.pqc.maven:pqc-sign-maven-plugin - as of 2026-05-12
32. github.com — 77626 - https://github.com/golang/go/issues/77626 - as of 2026-02-15
33. pkg.go.dev — mldsa - https://pkg.go.dev/crypto/mldsa - as of 2026-07-07
34. gpgfrontend.bktus.com — algorithms comparison - https://gpgfrontend.bktus.com/extra/algorithms-comparison/
35. developers.redhat.com — signing rpm packages using quantum resistant cryptography - https://developers.redhat.com/articles/2025/10/07/signing-rpm-packages-using-quantum-resistant-cryptography - as of 2026-02-27
36. Signing RPM packages using quantum-resistant cryptography - https://developers.redhat.com/articles/2025/10/07/signing-rpm-packages-using-quantum-resistant-cryptography - as of 2025-10-07
37. developer.android.com — features - https://developer.android.com/about/versions/17/features - as of 2026-07-13
38. APK signature scheme v3.2 | Android Open Source Project - https://source.android.com/docs/security/features/apksigning/v3-2 - as of 2026-07-29
39. techcommunity.microsoft.com — 4523370 - https://techcommunity.microsoft.com/blog/microsoft-security-blog/new-windows-features-to-secure-today%E2%80%99s-data-in-a-post-quantum-world/4523370 - as of 2026-07-14
40. encryptionconsulting.com — ml dsa support for your microsoft pki - https://www.encryptionconsulting.com/ml-dsa-support-for-your-microsoft-pki/ - as of 2026-06-30
41. Preparing the Windows ecosystem for next-generation code signing - https://support.microsoft.com/en-us/servicing/os/windows/docs/2026/08/next-generation-code-signing - as of 2026-08-20
42. encryptionconsulting.com — every code signing format codesign secure supports - https://www.encryptionconsulting.com/every-code-signing-format-codesign-secure-supports/ - as of 2026-08-07

43. blog.google — security for the quantum era implementing post quantum cryptography in android - https://blog.google/security/security-for-the-quantum-era-implementing-post-quantum-cryptography-in-android/ - as of 2026-03-25
44. support.google.com — 9842756 - https://support.google.com/googleplay/android-developer/answer/9842756
45. learn.microsoft.com — ml dsa overview - https://learn.microsoft.com/en-us/windows-server/identity/ad-cs/ml-dsa-overview - as of 2026-05-20
46. learn.microsoft.com — configure ml dsa certificate templates - https://learn.microsoft.com/en-us/windows-server/identity/ad-cs/configure-ml-dsa-certificate-templates - as of 2026-05-21
47. directaccess.richardhicks.com — microsoft ad cs adds post quantum cryptography support with ml dsa - https://directaccess.richardhicks.com/2026/05/18/microsoft-ad-cs-adds-post-quantum-cryptography-support-with-ml-dsa/ - as of 2026-05-18
48. cabforum.org — 2026 05 21 minutes of the server certificate working group - https://cabforum.org/2026/05/21/2026-05-21-minutes-of-the-server-certificate-working-group/ - as of 2026-05-21
49. accutivesecurity.com — code signing 2026 - https://accutivesecurity.com/code-signing-2026/ - as of 2026-05-06
50. techcommunity.microsoft.com — 4469093 - https://techcommunity.microsoft.com/blog/microsoft-security-blog/post-quantum-cryptography-apis-now-generally-available-on-microsoft-platforms/4469093 - as of 2025-11-12
51. security.apple.com — formal verification corecrypto - https://security.apple.com/blog/formal-verification-corecrypto/ - as of 2026-05-22
52. appleinsider.com — how apple turned to math to defend against next gen attacks on encryption - https://appleinsider.com/articles/26/05/26/how-apple-turned-to-math-to-defend-against-next-gen-attacks-on-encryption - as of 2026-05-27
53. helpnetsecurity.com — apple quantum resistant encryption open source - https://www.helpnetsecurity.com/2026/05/27/apple-quantum-resistant-encryption-open-source/ - as of 2026-05-27
54. windowsnews.ai — microsoft sets 2029 deadline for post quantum cryptography readiness across tls 13 code signing and .433564 - https://windowsnews.ai/article/microsoft-sets-2029-deadline-for-post-quantum-cryptography-readiness-across-tls-13-code-signing-and-.433564 - as of 2026-07-02
55. evertrust.io — hybrid post quantum certificates - https://evertrust.io/blog/hybrid-post-quantum-certificates/ - as of 2026-06-03
56. TrustFabric and Post-Quantum - https://blog.transparency.dev/trustfabric-and-post-quantum - as of 2026-07-10
57. [Feature Request] ML-DSA (FIPS 204) post-quantum signing and verification support - https://github.com/sigstore/cosign/issues/4960 - as of 2026-06-21
58. sigstore.dev and Rekor evolution - https://blog.sigstore.dev/rekor-evolution/ - as of 2026-06-28
59. github.blog — introducing artifact attestations now in public beta - https://github.blog/news-insights/product-news/introducing-artifact-attestations-now-in-public-beta/

60. openssf.org — introducing artifact attestations now in public beta - https://openssf.org/blog/2024/05/24/introducing-artifact-attestations-now-in-public-beta/
61. authorea-prod.literatumonline.com — v2 - https://authorea-prod.literatumonline.com/doi/full/10.22541/au.177153860.05574964/v2 - as of 2026-02-23
62. encryptionconsulting.com — the 2023 msi code signing data theft - https://www.encryptionconsulting.com/the-2023-msi-code-signing-data-theft/
63. learn.microsoft.com — when will azure devops ssh support post quantum ke - https://learn.microsoft.com/en-us/answers/questions/5898339/when-will-azure-devops-ssh-support-post-quantum-ke - as of 2026-05-22
64. docs.aws.amazon.com — mldsa - https://docs.aws.amazon.com/kms/latest/developerguide/mldsa.html
65. aws.amazon.com — aws private ca post quantum digital certificates - https://aws.amazon.com/about-aws/whats-new/2025/11/aws-private-ca-post-quantum-digital-certificates/ - as of 2025-11-10
66. aws.amazon.com — post quantum cryptography - https://aws.amazon.com/security/post-quantum-cryptography/
67. cloud.google.com — pqc in plaintext google clouds post quantum cryptography roadmap - https://cloud.google.com/blog/products/identity-security/pqc-in-plaintext-google-clouds-post-quantum-cryptography-roadmap
68. cloud.google.com — future proofing data integrity quantum safe digital signatures in cloud kms - https://cloud.google.com/blog/products/identity-security/future-proofing-data-integrity-quantum-safe-digital-signatures-in-cloud-kms
69. globalsecuritymag.com — announcing quantum safe digital signatures in cloud kms - https://www.globalsecuritymag.com/announcing-quantum-safe-digital-signatures-in-cloud-kms.html
70. developer.hashicorp.com — transit - https://developer.hashicorp.com/vault/api-docs/secret/transit
71. hashicorp.com — vault enterprise 1 19 reduces risk encryption updates automated root rotation - https://www.hashicorp.com/en/blog/vault-enterprise-1-19-reduces-risk-encryption-updates-automated-root-rotation
72. blog.sigstore.dev - https://blog.sigstore.dev/
73. github.com — signature envelope cose - https://github.com/notaryproject/notaryproject/blob/main/specs/signature-envelope-cose.md
74. Post-Quantum Artifact Signing in CI/CD: Migrating cosign … - https://www.systemshardening.com/articles/cicd/post-quantum-artifact-signing/ - as of 2026-05-08
75. quantumsecuritydefence.com — cloud kms pqc readiness aws azure gcp - https://quantumsecuritydefence.com/insights/cloud-kms-pqc-readiness-aws-azure-gcp/ - as of 2026-07-13
76. quantumsequrity.com — azure key vault post quantum - https://quantumsequrity.com/blog/azure-key-vault-post-quantum.md
77. learn.microsoft.com — about keys - https://learn.microsoft.com/en-us/azure/key-vault/keys/about-keys